

Mini-projet GLPI — Base de Données Répartie

ING2 GIA2 — Bases de Données Avancées (SIE / BDDR)

CY Tech — Année 2025–2026

Table des matières

1. Démarche de reverse engineering.....	4
1.1 Méthodologie	4
1.2 Périmètre fonctionnel retenu	4
1.3 Constatations sur la base GLPI.....	4
1.3.1 GLPI tourne sur MySQL / MariaDB.....	4
1.3.2 Un schéma de plus de 250 tables.....	5
1.3.3 Liaisons polymorphiques (itemtype / items_id)	5
1.3.4 Peu de clés étrangères déclaratives	5
1.3.5 Le multi-sites de GLPI n'est qu'un filtre applicatif.....	5
1.3.6 Indexation basique	5
1.4 Synthèse des problèmes identifiés.....	6
1.5 Choix de conception en réponse.....	6
Deux schémas Oracle séparés, reliés par DB link.....	6
Fragmentation horizontale par site_id.....	6
Tables de référence : propriétaire unique + vues matérialisées.....	6
Tablespaces DATA + IDX par site	7
Cluster sur DEVICE et PERIPHERAL	7
Schéma simplifié avec vraies FK.....	7
Tickets centralisés sur Cergy avec procédures PL/SQL	7
1.6 Bilan.....	7
2. Nouvelle modélisation de la base	8
2.1 Vue d'ensemble : 13 tables	8
2.2 Architecture BDDR — répartition Cergy / Pau	8
2.3 Diagramme entité-relation (ERD)	9
2.4 Schéma relationnel détaillé	9
Tables de référence (propriétaire = Pau).....	9
Tables de référence (propriétaire = Cergy).....	10
Tables locales (présentes sur Cergy et Pau).....	10
Table centralisée (sur Cergy uniquement).....	10

2.5 Justification de la normalisation	10
3. Implémentation des concepts avancés	11
3.1 Bases de Données Réparties (BDDR)	11
3.1.1 Database Links bidirectionnels	11
3.1.2 Vues matérialisées (REFRESH ON DEMAND)	11
3.1.3 Vues distantes simples (non matérialisées)	11
3.1.4 Procédures cross-site	12
3.2 Organisation physique : tablespaces et cluster	12
3.2.1 Séparation données / index	12
3.2.2 Cluster DEVICE / PERIPHERAL	12
3.3 Stratégie d'indexation	12
3.3.1 Index sur clés étrangères (catégorie A)	12
3.3.2 Index métier (catégorie B)	13
3.4 Utilisateurs et rôles	13
3.5 PL/SQL — Procédures principales	13
3.6 PL/SQL — Fonctions	14
3.7 PL/SQL — Triggers	14
Famille 1 : validation des références MV	14
Famille 2 : cohérence métier	14
Famille 3 : audit	15
3.8 PL/SQL — Curseurs explicites	15
3.9 Gestion d'erreurs centralisée	15
Table ERROR_LOG	15
Procédure PROC_LOG_ERROR (autonome)	15
Vue unifiée V_ERROR_LOG_ALL (côté Cergy)	16
3.10 Synthèse des objets PL/SQL	16
4. Résultats des tests de performance	16
4.1 Méthodologie	16
Jeu de test	16
Outil de mesure : EXPLAIN PLAN	17
Comment on compare « avec » et « sans » optimisation	17
Pourquoi pas de mesure de temps d'exécution ?	17
4.2 Test 1 — Efficacité des index	17
Sans index : FULL SCAN	17
Avec index unique	18
Conclusion test 1	19

4.3 Test 2 — Efficacité du cluster DEVICE / PERIPHERAL	19
Sans cluster (cluster désactivé par + 0).....	19
Avec cluster.....	20
Conclusion test 2.....	21
4.4 Test 3 — Vues matérialisées et résilience réseau.....	21
Sans MV : accès distant via DB link	22
Avec MV : accès local	22
Pourquoi le Cost est identique (3 vs 3) ?	23
Conclusion test 3.....	24
4.5 Test 4 — Bénéfice de la fragmentation horizontale	24
Sans fragmentation : requête nationale	24
Avec fragmentation : requête purement locale	25
Conclusion test 4.....	26
4.6 Synthèse des gains.....	26
Limites de la démarche	27

1. Démarche de reverse engineering

1.1 Méthodologie

Pour ce projet, nous avons commencé par étudier la base de données de GLPI à partir du dépôt officiel `glpi-project/glpi` sur GitHub et de la documentation associée. L'idée n'était pas de tout reprendre — GLPI contient plus de **250 tables** — mais de comprendre comment la base est organisée et d'en repérer les limites pour notre cas d'usage : la gestion du parc informatique de CY Tech sur deux sites (Cergy et Pau).

Notre démarche a été simple :

- **regarder les tables principales de GLPI** liées au parc (ordinateurs, périphériques, utilisateurs, lieux, tickets) ;
- **comprendre les choix de modélisation** : nommage des tables et des colonnes, gestion des liaisons, notion d'« entités » pour le multi-tenant ;
- **comparer avec les attentes du sujet** (multi-sites Cergy / Pau, BDDR, PL/SQL, optimisation), et noter les points qui ne collent pas.

1.2 Périmètre fonctionnel retenu

GLPI fait beaucoup plus que de la gestion de parc : ITSM, contrats, projets, base de connaissances, etc. Nous avons choisi de nous concentrer uniquement sur ce qui est demandé dans le sujet :

- **les matériels** : ordinateurs et périphériques (`glpi_computers`, `glpi_peripherals`, `glpi_monitors`) ;
- **les utilisateurs** et leurs rôles (`glpi_users`, `glpi_profiles`) ;
- **la localisation** : sites, bâtiments, salles (`glpi_locations`, `glpi_entities`) ;
- **les systèmes d'exploitation** (`glpi_operatingsystems`) ;
- **les tickets de maintenance** (`glpi_tickets`).

Tout ce qui sort du périmètre (financier, projets, connaissances...) a été mis de côté.

1.3 Constatations sur la base GLPI

En étudiant la base, nous avons relevé plusieurs points qui posent problème dès qu'on passe sur Oracle et qu'on veut faire du multi-sites pour de vrai.

1.3.1 GLPI tourne sur MySQL / MariaDB

GLPI est prévu pour MySQL / MariaDB. Cela veut dire qu'il n'utilise **aucune des fonctionnalités Oracle** que le sujet nous demande d'employer :

- pas de tablespaces utilisateurs pour séparer données et index ;
- pas de clusters pour rapprocher physiquement des tables liées ;
- pas de Database Link entre bases : la communication entre deux instances passe par la couche applicative PHP ;
- pas de PL/SQL : toute la logique métier est dans le code PHP.

Conclusion : si on veut vraiment utiliser ce qui a été vu en cours (tablespaces, clusters, BDDR, PL/SQL), il faut refaire la base.

1.3.2 Un schéma de plus de 250 tables

La doc indique que GLPI contient **plus de 250 tables**. C'est cohérent avec un outil qui couvre énormément de fonctionnalités, mais pour nous c'est ingérable : impossible de modéliser, d'indexer et de tester 250 tables en quelques semaines. On a donc choisi un **schéma simplifié de 13 tables** qui couvre exactement le périmètre fixé.

1.3.3 Liaisons polymorphiques (itemtype / items_id)

Plutôt que d'utiliser des clés étrangères classiques, GLPI utilise souvent un système de liaison « polymorphique » : une table de liaison contient deux colonnes ensemble — items_id (l'identifiant) et itemtype (le type, sous forme de texte : 'Computer', 'Printer', etc.).

C'est pratique côté code PHP (on peut tout lier à tout), mais ça pose deux problèmes pour nous :

- **on ne peut pas mettre de vraie clé étrangère** : la base ne peut pas vérifier que l'identifiant pointe vers le bon type d'objet ;
- **les jointures deviennent compliquées** parce que la cible dépend d'une colonne de texte.

Dans notre refonte, on a remplacé ça par des **FK classiques** : par exemple, PERIPHERAL.assigned_device_id pointe directement sur DEVICE.device_id.

1.3.4 Peu de clés étrangères déclaratives

GLPI ne déclare presque pas de FOREIGN KEY dans la base. L'intégrité est **vérifiée par le code PHP** (au moment des INSERT / UPDATE / DELETE), pas par la base elle-même. Ça veut dire que si quelqu'un attaque la base directement en SQL, il peut laisser des données incohérentes.

Sur Oracle, on peut faire mieux : on déclare **toutes les FK possibles**, et c'est la base qui garantit l'intégrité.

1.3.5 Le multi-sites de GLPI n'est qu'un filtre applicatif

GLPI gère le multi-sites avec une table glpi_entities et une colonne entities_id présente sur la plupart des tables. Mais ce n'est qu'un **filtre côté application** : toutes les données se trouvent dans la même base, sur le même serveur. Il n'y a aucune séparation physique entre les sites.

Or le sujet demande explicitement de **gérer la distribution des données entre Cergy et Pau**. Donc il faut faire une vraie répartition, pas juste un filtre.

1.3.6 Indexation basique

La doc GLPI recommande d'indexer les clés étrangères et quelques colonnes souvent filtrées, mais ça reste assez générique. Sur Oracle, on peut aller plus loin : choisir les index

en fonction des requêtes réelles, les ranger dans un tablespace dédié, et regarder les plans d'exécution pour vérifier qu'ils sont bien utilisés.

1.4 Synthèse des problèmes identifiés

En résumé, nous avons identifié six problèmes principaux sur lesquels notre refonte doit apporter une réponse :

#	Problème identifié sur GLPI	Notre réponse
P1	SGBD MySQL : pas de tablespace, cluster, ni BDDR natif	Refaire la base sur Oracle en utilisant ces fonctionnalités
P2	Schéma à 250+ tables	Schéma simplifié à 13 tables centré sur le périmètre
P3	Liaisons polymorphiques itemtype/items_id	Remplacement par des FK classiques
P4	Peu de FK déclarées, intégrité gérée en PHP	Toutes les FK et CHECK possibles déclarées en SQL
P5	Multi-sites = simple filtre applicatif	Vraie fragmentation horizontale + DB link entre Cergy et Pau
P6	Indexation générique	Index choisis en fonction des requêtes réelles, tablespace IDX séparé

1.5 Choix de conception en réponse

À partir de ces constats, nous avons fait les choix d'architecture suivants. Chaque choix répond à un ou plusieurs des problèmes ci-dessus.

Deux schémas Oracle séparés, reliés par DB link

→ Répond à **P1** et **P5**. Nous avons créé deux schémas distincts dans la même instance Oracle : CYTECH_CERGY et CYTECH_PAU. Chacun a ses propres tablespaces et ses propres tables. Les deux schémas communiquent par deux Database Links (LNK_PAU et LNK_CERGY) ce qui permet à un site d'aller lire ou écrire sur l'autre quand il en a besoin.

Fragmentation horizontale par site_id

→ Répond à **P5**. Les tables métier (PERSON, DEVICE, PERIPHERAL, BUILDING, ROOM) existent sur les deux sites, mais chaque site ne contient que ses propres lignes : site_id = 1 pour Cergy, site_id = 2 pour Pau. C'est garanti par des contraintes CHECK dans le DDL, donc impossible d'insérer une ligne « du mauvais site » par erreur.

Tables de référence : propriétaire unique + vues matérialisées

→ Répond à **P1** et **P5**. Pour les tables de référence (celles qui contiennent les mêmes données partout), on a choisi un propriétaire unique :

- **Pau** possède SITE et PERSON_ROLE ;
- **Cergy** possède DEVICE_TYPE, PERIPHERAL_TYPE, OS_FAMILY, OS_VERSION.

Chaque site dispose en lecture des tables de l'autre via des **vues matérialisées REFRESH ON DEMAND**. On a choisi cette option plutôt que des triggers de réplication temps réel

parce que ces données changent très rarement (2-3 fois par an) : un mécanisme temps réel serait inutilement complexe et risquerait de créer des blocages cross-site.

Tablespaces DATA + IDX par site

→ Répond à **P1 et P6**. Chaque site a deux tablespaces : un pour les données (DATA_CERGY, DATA_PAU) et un pour les index (IDX_CERGY, IDX_PAU). Cette séparation permet d'organiser le stockage plus proprement et de répartir les accès disque.

Cluster sur DEVICE et PERIPHERAL

→ Répond à **P6**. On utilise souvent la même requête : « lister un device et tous ses périphériques associés ». Pour accélérer ça, on a placé DEVICE et PERIPHERAL dans un même cluster Oracle indexé sur device_id. Résultat : les lignes liées sont stockées physiquement à proximité, ce qui réduit les lectures disque sur cette jointure.

Schéma simplifié avec vraies FK

→ Répond à **P2, P3 et P4**. Notre schéma à 13 tables n'utilise que des relations classiques : chaque colonne qui pointe vers une autre table est déclarée comme FOREIGN KEY. Les valeurs des colonnes de statut (device_status, ticket_status, etc.) sont contrôlées par des CHECK IN (...). L'intégrité est gérée par la base, pas par l'application.

Cas particulier : Oracle n'autorise pas de FK vers une vue matérialisée ni vers une table distante (via DB link). Pour les colonnes concernées (par exemple site_id dans MAINTENANCE_TICKET), on compense avec des CHECK et avec des procédures PL/SQL qui vérifient l'existence de la donnée avant l'insertion.

Tickets centralisés sur Cergy avec procédures PL/SQL

→ Répond à **P5**. La table MAINTENANCE_TICKET n'existe que sur Cergy, qui joue le rôle de site central pour la maintenance. Pau ouvre ses tickets via la procédure PROC_OPEN_TICKET_PAU, qui insère à distance via le DB link. Côté Cergy, PROC_CREATE_TICKET vérifie d'abord que le device_id concerné existe bien sur le bon site avant d'insérer.

1.6 Bilan

GLPI n'est pas mal conçu, mais il est fait pour un autre contexte : un produit MySQL open source où l'intégrité est gérée par PHP. Notre travail n'est pas de « corriger » GLPI, mais de **repenser le modèle sur Oracle** pour utiliser réellement ce qui a été vu en cours et pour gérer correctement la séparation entre les deux sites.

Les six problèmes que nous avons identifiés sont la base de tous les choix de conception qui suivent dans le rapport.

2. Nouvelle modélisation de la base

Cette section présente la modélisation que nous avons retenue : la liste des tables et leurs relations (schéma relationnel), le diagramme entité-relation, et la vue d'ensemble de l'architecture répartie entre Cergy et Pau.

2.1 Vue d'ensemble : 13 tables

Notre modèle compte 13 tables réparties en trois catégories selon leur rôle et leur emplacement physique :

- **6 tables de référence** (peu volumineuses, modifiées très rarement, partagées entre les deux sites) : SITE, PERSON_ROLE, DEVICE_TYPE, PERIPHERAL_TYPE, OS_FAMILY, OS_VERSION.
- **6 tables locales** (présentes sur chaque site, fragmentées par site_id) : BUILDING, ROOM, PERSON, DEVICE, PERIPHERAL, DEVICE_ASSIGNMENT.
- **1 table centralisée** (présente uniquement sur Cergy, gère le SAV des 2 sites) : MAINTENANCE_TICKET.

2.2 Architecture BDDR — répartition Cergy / Pau

Le schéma ci-dessous résume comment les 13 tables sont distribuées entre les deux schémas Oracle et comment elles communiquent via les Database Links.

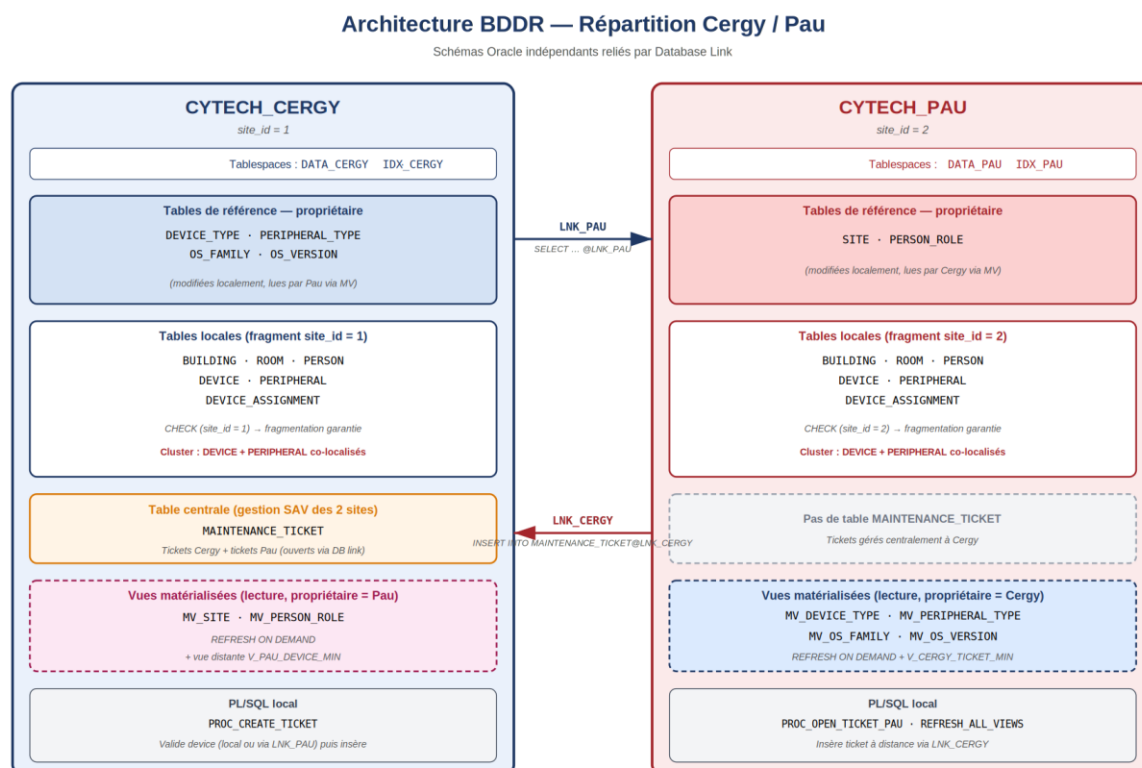


Figure 1 — Architecture BDDR : schémas CYTECH_CERGY et CYTECH_PAU, tablespaces, fragmentation, MV de réplication et tickets centralisés.

Trois points clés à retenir sur ce schéma :

- **Chaque site est propriétaire de certaines tables de référence** (Pau pour SITE et PERSON_ROLE, Cergy pour les types et OS) et les expose à l'autre site via des vues matérialisées en REFRESH ON DEMAND.
- **Les tables locales sont fragmentées horizontalement** : Cergy contient les lignes `site_id = 1`, Pau les `site_id = 2`, garanti par contraintes CHECK.
- **MAINTENANCE_TICKET** est centralisée sur Cergy : Pau y insère à distance via LNK_CERGY (procédure PROC_OPEN_TICKET_PAU).

2.3 Diagramme entité-relation (ERD)

Le diagramme suivant montre les 13 tables avec leurs principaux attributs et toutes les relations qui les lient. Les couleurs reprennent celles du schéma d'architecture : rouge = tables propriété de Pau, bleu foncé = propriété de Cergy, vert = grosses tables placées dans le cluster, gris = autres tables locales, orange = table centralisée.

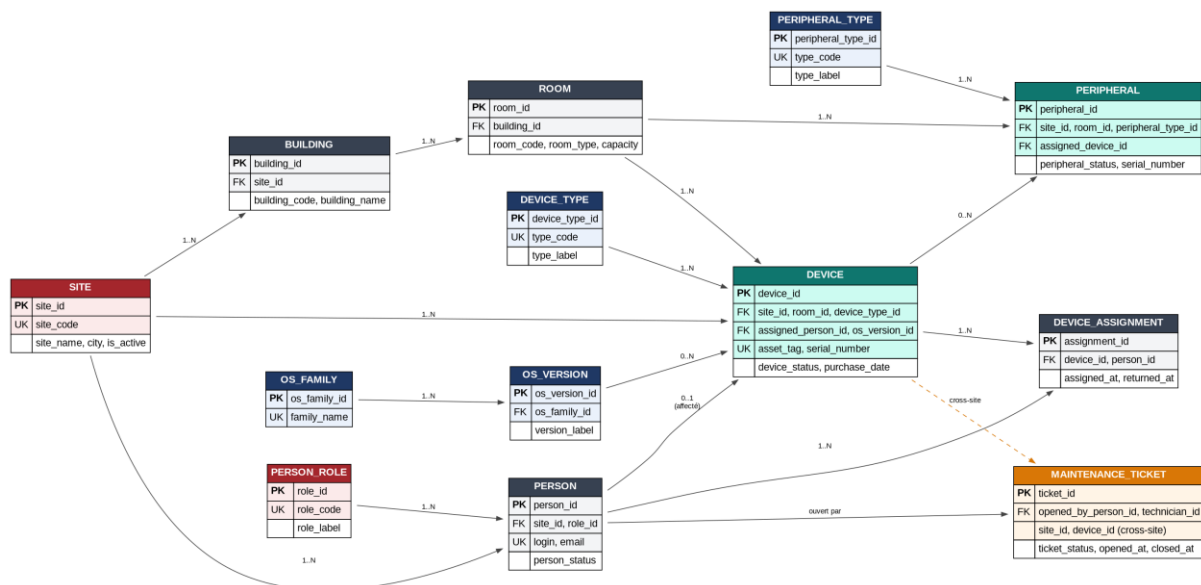


Figure 2 — Diagramme entité-relation des 13 tables. Les flèches indiquent les clés étrangères et leurs cardinalités. La flèche pointillée orange représente la relation cross-site (FK non déclarée, compensée par CHECK + procédure).

2.4 Schéma relationnel détaillé

Pour la lisibilité fine, voici le schéma relationnel sous forme textuelle. Les clés primaires sont soulignées (PK), les clés étrangères sont préfixées par #, et les contraintes UNIQUE sont notées (UK).

Tables de référence (propriétaire = Pau)

SITE	(PK <u>site_id</u> , UK site_code, site_name, city, is_active)
PERSON_ROLE	(PK <u>role_id</u> , UK role_code, role_label)

Tables de référence (propriétaire = Cergy)

DEVICE_TYPE	(PK device_type_id, UK type_code, type_label)
PERIPHERAL_TYPE	(PK peripheral_type_id, UK type_code, type_label)
OS_FAMILY	(PK os_family_id, UK family_name)
OS_VERSION	(PK os_version_id, # os_family_id, version_label)

Tables locales (présentes sur Cergy et Pau)

BUILDING	(PK building_id, # site_id, building_code, building_name)
ROOM	(PK room_id, # building_id, room_code, room_type, capacity)
PERSON	(PK person_id, # site_id, # role_id, UK login, UK email, person_status)
DEVICE	(PK device_id, # site_id, # room_id, # assigned_person_id, # device_type_id, # os_version_id, UK asset_tag, UK serial_number, purchase_date, device_status)
PERIPHERAL	(PK peripheral_id, # site_id, # room_id, # assigned_device_id, # peripheral_type_id, peripheral_status, serial_number)
DEVICE_ASSIGNMENT	(PK assignment_id, # device_id, # person_id, assigned_at, returned_at)

Table centralisée (sur Cergy uniquement)

MAINTENANCE_TICKET	(PK ticket_id, # opened_by_person_id, # technician_id, site_id (CHECK IN 1,2), device_id (cross-site), issue_label, ticket_status, opened_at, closed_at)
--------------------	--

Remarques importantes :

- Les colonnes # représentent des **vraies FK Oracle** déclaratives (avec contrainte FOREIGN KEY).
- Sur MAINTENANCE_TICKET, les colonnes site_id et device_id **n'ont pas de FK déclarative** parce qu'elles peuvent référencer une donnée Pau (table distante via DB link, FK Oracle interdite). L'intégrité est assurée par les contraintes CHECK et par la procédure PROC_CREATE_TICKET qui valide l'existence avant l'insertion.
- Les statuts (device_status, peripheral_status, ticket_status, person_status) sont tous protégés par des CHECK IN (...) pour empêcher toute valeur non prévue.

2.5 Justification de la normalisation

Notre modèle respecte la 3e forme normale (3NF) : chaque attribut non-clé dépend uniquement de la clé primaire, et il n'y a pas de dépendance transitive. Quelques exemples concrets :

- **Type de device séparé de DEVICE** : on a une table DEVICE_TYPE plutôt qu'une colonne texte dans DEVICE. Permet d'ajouter un type sans toucher au DDL et garantit la cohérence des libellés.

- **Famille et version d'OS séparées** : OS_FAMILY (Windows, Linux...) puis OS_VERSION (Windows 11, Ubuntu 24.04...). Évite la duplication de la famille sur chaque ligne de version.
- **Affectations historisées dans une table dédiée** : DEVICE_ASSIGNMENT conserve l'historique des prêts (avec assigned_at/returned_at), alors que DEVICE.assigned_person_id donne juste l'affectation courante. Sans ça, on perd l'historique à chaque réaffectation.

On ne va pas plus loin (BCNF, 4NF, etc.) parce que notre cas d'usage ne le justifie pas : nos dépendances fonctionnelles sont simples et la 3NF suffit largement.

3. Implémentation des concepts avancés

Cette section présente comment nous avons mis en œuvre les notions vues en cours dans notre base. Pour chaque concept, on explique brièvement ce qu'on a fait, pourquoi, et où ça se trouve dans nos scripts.

3.1 Bases de Données Réparties (BDDR)

C'est le cœur du projet. On a fait quatre choses pour répartir réellement les données entre Cergy et Pau.

3.1.1 Database Links bidirectionnels

Chaque site a un Database Link qui pointe vers l'autre : LNK_PAU côté Cergy, et LNK_CERGY côté Pau. Un Database Link, c'est un « pont » entre deux schémas Oracle : une fois créé, on peut faire SELECT ... FROM table@LNK_PAU comme si la table était locale. C'est ce qui rend possible toute la suite : MV, vues distantes, procédures cross-site.

3.1.2 Vues matérialisées (REFRESH ON DEMAND)

Une vue matérialisée stocke physiquement le résultat d'une requête sur une table distante. On l'a utilisée pour les tables de référence dont chaque site n'est pas propriétaire :

- **côté Cergy** : MV_SITE, MV_PERSON_ROLE (qui pointent vers Pau) ;
- **côté Pau** : MV_DEVICE_TYPE, MV_PERIPHERAL_TYPE, MV_OS_FAMILY, MV_OS_VERSION (qui pointent vers Cergy).

L'option REFRESH ON DEMAND veut dire que la mise à jour se déclenche manuellement, via DBMS_MVIEW.REFRESH(...). On a aussi écrit une procédure REFRESH_ALL_VIEWS côté Pau qui rafraîchit les 4 MV d'un coup, pour simplifier la maintenance.

3.1.3 Vues distantes simples (non matérialisées)

Quand on veut juste consulter une donnée distante en temps réel, sans stocker localement, on utilise une vue classique posée sur la table distante :

- V_PAU_DEVICE_MIN (côté Cergy) : liste minimale des devices de Pau ;
- V_CERGY_TICKET_MIN (côté Pau) : liste des tickets ouverts à Cergy ;

- V_ERROR_LOG_PAU et V_ERROR_LOG_ALL (côté Cergy) : consultation des logs d'erreurs Pau, et union des deux sites.

La différence avec une MV : on ne stocke rien, on relit à chaque requête. C'est moins performant mais toujours à jour, donc adapté aux données qui changent souvent (tickets, logs).

3.1.4 Procédures cross-site

On a deux procédures qui font des opérations distribuées :

- **PROC_CREATE_TICKET** (côté Cergy) : crée un ticket de maintenance après avoir vérifié que le device existe bien sur le site indiqué. Si le site est Cergy, vérification locale ; si c'est Pau, vérification via DEVICE@LNK_PAU.
- **PROC_OPEN_TICKET_PAU** (côté Pau) : appelée par un manager Pau pour ouvrir un ticket. Elle vérifie d'abord que le device existe bien à Pau (localement), puis **insère à distance** dans MAINTENANCE_TICKET@LNK_CERGY en récupérant le NEXTVAL de la séquence Cergy également via le DB link.

Ces deux procédures sont l'illustration concrète d'une transaction distribuée : on lit sur un site et on écrit sur l'autre dans la même unité logique.

3.2 Organisation physique : tablespaces et cluster

3.2.1 Séparation données / index

Chaque site dispose de deux tablespaces : un pour les données (DATA_CERGY, DATA_PAU) et un pour les index (IDX_CERGY, IDX_PAU). Toutes les CREATE TABLE et toutes les CREATE INDEX précisent leur tablespace cible, ce qui permet une organisation propre du stockage et une analyse plus simple de la volumétrie.

3.2.2 Cluster DEVICE / PERIPHERAL

On a créé un cluster Oracle (cl_device_periph côté Cergy, cl_device_periph_pau côté Pau) qui regroupe physiquement DEVICE et PERIPHERAL. Concrètement, Oracle stocke côte à côte les lignes qui ont la même valeur de device_id : un device et tous ses périphériques se retrouvent dans les mêmes blocs disque. La jointure « lister un device et ses périphériques » devient donc beaucoup plus rapide qu'avec deux tables séparées.

Le cluster est obligatoirement accompagné d'un index de cluster, qui doit exister avant toute insertion. C'est ce qu'on déclare juste après le CREATE CLUSTER.

3.3 Stratégie d'indexation

On a deux catégories d'index, regroupées dans le script 07_indexes.sql :

3.3.1 Index sur clés étrangères (catégorie A)

Sur Oracle, une colonne FK non indexée entraîne un verrou sur toute la table fille à chaque modification de la table mère (notion vue en CM3 sur les verrous). On indexe donc systématiquement toutes les colonnes FK qui ne le sont pas déjà par la PK : par exemple

MAINTENANCE_TICKET.opened_by_person_id, MAINTENANCE_TICKET.technician_id, DEVICE.os_version_id, PERIPHERAL.room_id, etc.

Exception : on **n'indexe pas** site_id seul, car la contrainte CHECK (site_id = X) fait qu'il n'y a qu'une seule valeur possible par site. Un index dessus serait inutile (cardinalité = 1).

3.3.2 Index métier (catégorie B)

Ces index ciblent des requêtes précises qu'on utilise vraiment :

- **IDX_TICKET_STATUS_OPEN** : index composite (ticket_status, opened_at) qui sert la vue V_ACTIVE_TICKETS (filtrage par statut + tri par date).
- **IDX_DEVICE_STATUS** : pour les rapports « devices en réparation ou retirés ».
- **IDX_ASSIGN_ACTIVE** : index fonctionnel sur CASE WHEN returned_at IS NULL THEN device_id END. Comme Oracle exclut les NULL des index B-tree, cet index ne contient que les affectations actives, donc il est beaucoup plus petit qu'un index plein.
- **IDX_PERSON_STATUS_ROLE** : composite pour les rapports « personnes actives par rôle ».

Après création, on lance DBMS_STATS.GATHER_SCHEMA_STATS pour que l'optimiseur Oracle utilise réellement ces index dans ses plans d'exécution. On peut vérifier l'utilisation avec EXPLAIN PLAN (analyses détaillées dans la partie tests de performance).

3.4 Utilisateurs et rôles

Pour gérer les droits d'accès, on a mis en place une hiérarchie de trois rôles par site. Chaque rôle hérite du précédent :

- **R_xxx_READ** : lecture (SELECT) sur toutes les tables et vues du site, y compris les MV et vues distantes ;
- **R_xxx_MANAGER** : hérite de READ + INSERT/UPDATE sur les tables métier (PERSON, DEVICE, etc.) + EXECUTE sur les procédures de saisie ;
- **R_xxx_ADMIN** : hérite de MANAGER + DELETE et droits complets sur les tables de référence.

On a aussi créé 6 utilisateurs de test (3 par site) pour valider concrètement que les droits sont bien restrictifs : un U_CERGY_READ ne peut pas insérer, un U_CERGY_MANAGER ne peut pas supprimer, etc. Cette approche par rôles est conforme à ce qui est vu en cours sur la gestion fine des accès.

3.5 PL/SQL — Procédures principales

En plus des deux procédures cross-site déjà décrites (3.1.4), on a :

- **PROC_GENERATE_DATA** : procédure paramétrée qui insère des données aléatoires de test (personnes, devices, périphériques, tickets pour Cergy). Elle utilise DBMS_RANDOM pour générer les valeurs, et chaque exécution produit un jeu de données différent. C'est cette procédure qui nous permet de faire des tests de performance représentatifs.
- **REFRESH_ALL_VIEWS** (côté Pau) : rafraîchit les 4 vues matérialisées en une seule commande, via une boucle FOR sur une collection de noms de MV.

3.6 PL/SQL — Fonctions

Trois fonctions qui simplifient l'écriture des requêtes courantes :

- **FCT_DEVICE_AGE(p_device_id)** : renvoie l'âge d'un device en mois depuis sa date d'achat. Utilisable directement dans une requête : `SELECT asset_tag, FCT_DEVICE_AGE(device_id) FROM DEVICE.`
- **FCT_TICKET_DURATION_DAYS(p_ticket_id)** (Cergy uniquement) : durée en jours entre l'ouverture et la clôture d'un ticket. Si le ticket est encore ouvert, renvoie la durée écoulée depuis l'ouverture.
- **FCT_DEVICE_HAS_ACTIVE_TICKET(p_device_id)** (Cergy uniquement) : renvoie 1 si le device a un ticket OPEN ou IN_PROGRESS, 0 sinon. Le retour est NUMBER et non BOOLEAN parce qu'Oracle n'accepte pas BOOLEAN dans les requêtes SQL.

3.7 PL/SQL — Triggers

On a organisé les triggers en trois familles :

Famille 1 : validation des références MV

Comme Oracle interdit les FK vers une vue matérialisée, ces triggers compensent en vérifiant manuellement que la valeur insérée existe bien dans la MV correspondante :

- **TRG_VALIDATE_ROLE_CERGY** : vérifie que `PERSON.role_id` existe bien dans `MV_PERSON_ROLE` ;
- **TRG_VALIDATE_DEVICE_TYPE_PAU**, **TRG_VALIDATE_DEVICE_OS_PAU**, **TRG_VALIDATE_PERIPH_TYPE_PAU** : pareil pour les références de Pau vers les MV qui viennent de Cergy.

En cas de référence invalide, le trigger lève une erreur applicative (`RAISE_APPLICATION_ERROR`) et logue l'incident dans `ERROR_LOG`.

Famille 2 : cohérence métier

Ces triggers imposent des règles du domaine qui ne peuvent pas s'exprimer en CHECK :

- **TRG_DEVICE_RETIRED_GUARD** : refuse de passer un device en RETIRED s'il est encore assigné à quelqu'un.
- **TRG_DEVICE_ASSIGN_ACTIVE** : refuse d'assigner un device à une personne INACTIVE.
- **TRG_TICKET_CLOSE_AUTO** : (i) interdit toute modification d'un ticket déjà CLOSED ; (ii) renseigne automatiquement `closed_at = SYSDATE` quand le statut passe à CLOSED.
- **TRG_ASSIGN_RETURN_SYNC** : quand une affectation est rendue (`returned_at` renseigné), libère automatiquement `DEVICE.assigned_person_id`.
- **TRG_PERSON_DEACTIVATE** : refuse de désactiver une personne qui a encore du matériel assigné ou des affectations actives.

Famille 3 : audit

TRG_AUDIT_DEVICE trace tout INSERT / UPDATE / DELETE sur DEVICE dans la table DEVICE_HISTORY (ancienne et nouvelle valeur de status, person, room, plus l'utilisateur et l'horodatage). Le trigger est en mode « best-effort » : si l'audit échoue (par exemple tablespace plein), on logue l'échec mais on ne bloque pas l'opération métier. On préfère perdre une ligne d'audit que de bloquer le parc.

3.8 PL/SQL — Curseurs explicites

On a trois procédures qui utilisent des curseurs explicites, chacune illustrant un pattern différent :

- **PROC_REPORT_PARC_BY_BUILDING** : **3 curseurs imbriqués** (BUILDING → ROOM → DEVICE) qui produisent un rapport hiérarchique du parc. C'est le pattern « curseur dans curseur » vu en CM6.
- **PROC_CLEANUP_ORPHAN_PERIPH** : utilise un curseur FOR UPDATE avec WHERE CURRENT OF pour libérer les périphériques dont le device a été RETIRED ou supprimé. Le FOR UPDATE verrouille chaque ligne avant la modification, WHERE CURRENT OF modifie la ligne courante sans re-chercher la clé primaire.
- **PROC_DETECT_AGING_TICKETS(p_threshold_days)** : curseur **paramétré** qui sélectionne les tickets ouverts depuis plus de N jours et logue chaque ligne dans ERROR_LOG avec app_code = 'TICKET_AGING' (utile pour générer des alertes archivables).

On a aussi PROC_RECONCILE_ASSIGNMENTS qui sert d'outil de diagnostic : il croise DEVICE_ASSIGNMENT et DEVICE.assigned_person_id pour repérer d'éventuelles incohérences. En régime normal, TRG_ASSIGN_RETURN_SYNC maintient la cohérence donc cette procédure doit retourner 0.

3.9 Gestion d'erreurs centralisée

Pour ne pas perdre la trace des erreurs et des alertes métier, on a mis en place un système de journalisation persistante sur chaque site.

Table ERROR_LOG

Chaque site a sa propre table ERROR_LOG qui stocke : code SQL, message d'erreur, nom du module fautif, utilisateur, contexte (paramètres) et call stack (DBMS_UTILITY.FORMAT_ERROR_BACKTRACE). Trois index la couvrent : sur error_ts (consultation chronologique), module_name (« quelle procédure plante le plus »), et app_code (filtrage par type d'erreur).

Procédure PROC_LOG_ERROR (autonome)

C'est la procédure clé du système. Elle utilise PRAGMA AUTONOMOUS_TRANSACTION, ce qui veut dire qu'elle s'exécute dans une transaction indépendante : **même si la transaction principale fait ROLLBACK, le log est conservé**. Sans ça, on perdrait toutes les traces des

erreurs ayant déclenché un rollback, alors que ce sont justement les plus importantes à analyser.

Deuxième détail important : le WHEN OTHERS THEN NULL final dans PROC_LOG_ERROR. Si la journalisation elle-même échoue (par exemple parce que le tablespace est plein), on ne veut pas faire échouer toute l'application. On préfère perdre le log plutôt que bloquer l'utilisateur.

Vue unifiée V_ERROR_LOG_ALL (côté Cergy)

Pour qu'un admin Cergy puisse consulter d'un coup les erreurs des deux sites sans se reconnecter, on a une vue V_ERROR_LOG_ALL qui fait un UNION ALL entre la table locale ERROR_LOG et la table distante ERROR_LOG@LNK_PAU. C'est aussi un bon exemple d'usage des vues distantes (non matérialisées) parce qu'on veut voir les erreurs en temps réel.

3.10 Synthèse des objets PL/SQL

Type	Nombre	Exemples
Procédures cross-site	2	PROC_CREATE_TICKET, PROC_OPEN_TICKET_PAU
Procédures métier / utilitaires	4+	PROC_GENERATE_DATA, proc_log_error, REFRESH_ALL_VIEWS, proc_cleanup_orphan_periph
Fonctions	3	fct_device_age, fct_ticket_duration_days, fct_device_has_ticket
Triggers	13	trg_check_role_cergy, trg_device_retired_guard, trg_assign_active, trg_ticket_close_auto, trg_assign_return_sync, trg_person_deactivate, trg_audit_device + versions Pau
Procédures à curseurs	2	proc_report_parc (curseurs imbriqués), proc_detect_aging_tickets
Vues matérialisées	6	MV_SITE, MV_PERSON_ROLE (Cergy) + 4 côté Pau
Vues distantes	4	V_PAU_DEVICE_MIN, V_CERGY_TICKET_MIN, V_NETWORK_TOPOLOGY, V_ERROR_LOG_ALL

4. Résultats des tests de performance

Cette section présente les mesures que nous avons faites pour valider que nos choix d'architecture (index, cluster, vues matérialisées, fragmentation) apportent réellement des gains de performance par rapport à une base sans ces optimisations.

4.1 Méthodologie

Jeu de test

Nous avons généré un jeu de données conséquent avec la procédure PROC_GENERATE_DATA, appelée avec les paramètres (5000, 20000, 5000) côté Cergy. Soit **5 000 personnes, 20 000 devices et 5 000 tickets**. On a ensuite recalculé les

statistiques de la base avec DBMS_STATS.GATHER_SCHEMA_STATS pour que l'optimiseur Oracle prenne en compte cette volumétrie.

Outil de mesure : EXPLAIN PLAN

Pour chaque test, on active SET AUTOTRACE TRACEONLY EXPLAIN et on lance la requête. Oracle affiche alors le **plan d'exécution** : les étapes prévues (FULL SCAN, INDEX SCAN, HASH JOIN, etc.) et le Cost calculé par l'optimiseur. Le Cost est une estimation de l'effort CPU et I/O nécessaire, indépendante du temps réel d'exécution. C'est la métrique de référence utilisée par Oracle pour choisir entre plusieurs plans possibles.

Comment on compare « avec » et « sans » optimisation

Le sujet demande une comparaison avec « l'ancienne base » (GLPI MySQL). On n'a pas pu faire tourner une instance GLPI MySQL en parallèle pour chronométrer dans des conditions identiques. À la place, on a choisi une approche pragmatique : on compare, sur notre base Oracle, le comportement avec nos optimisations actives vs le comportement quand on les désactive. Concrètement :

- **pour l'index** : on lance deux requêtes sur DEVICE, l'une qui force un FULL SCAN (filtre non indexé sur device_status), l'autre qui utilise l'index unique sur asset_tag ;
- **pour le cluster** : on désactive le cluster sur la même requête en ajoutant + 0 sur les colonnes de jointure (technique d'opération neutre qui empêche Oracle d'utiliser l'index de cluster et le force à faire un HASH JOIN avec FULL SCAN — comportement équivalent à une base sans cluster) ;
- **pour les MV** : on lance la même requête une fois avec accès distant (via @LNK_PAU) et une fois sur les MV locales ;
- **pour la fragmentation** : on lance une requête UNION ALL Cergy + Pau (équivalent à une base centralisée non répartie) vs une requête purement locale (équivalent à notre architecture fragmentée).

Pourquoi pas de mesure de temps d'exécution ?

Sur un jeu de 20 000 lignes dans un conteneur Docker local, le temps d'exécution est négligeable dans tous les cas (de l'ordre du centième de seconde). Pour observer une **différence de temps exploitable**, il faudrait plusieurs millions de lignes — ce qui dépasse les capacités de nos machines pour la génération. On compare donc les **ressources estimées (Cost, Bytes traités, type d'opération)** plutôt que le temps. C'est la métrique recommandée par Oracle pour ce type d'analyse.

4.2 Test 1 — Efficacité des index

On compare deux requêtes sur la même table DEVICE (20 000 lignes).

Sans index : FULL SCAN

```
SELECT * FROM DEVICE WHERE device_status = 'IN_REPAIR';
```

```
SQL>
SQL> SELECT * FROM DEVICE WHERE device_status = 'IN_REPAIR';
```

Execution Plan

Plan hash value: 1281438832

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		4973	573K	376 (1)	00:00:01
* 1	TABLE ACCESS FULL	DEVICE	4973	573K	376 (1)	00:00:01

Predicate Information (identified by operation id):

1 - filter("DEVICE_STATUS"='IN_REPAIR')

screen index nul

Figure 3 — Plan d'exécution sans index utilisable : TABLE ACCESS FULL, Cost = 376, 573 Ko traités.

Oracle parcourt la table entière car le filtre sur device_status ne peut pas tirer parti de notre stratégie d'indexation (la cardinalité est faible : seulement 4 valeurs possibles). **Cost = 376**, 4 973 lignes retournées, 573 Ko traités.

Avec index unique

```
SELECT * FROM DEVICE WHERE asset_tag = 'CGY-AUTO-500';
```

```
SQL> SELECT * FROM DEVICE WHERE asset_tag = 'CGY-AUTO-500';
```

Execution Plan

Plan hash value: 2320806943

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	118	(0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	DEVICE	1	118	(0)	00:00:01
* 2	INDEX UNIQUE SCAN	UK_DEVICE_ASSET_TAG	1		(0)	00:00:01

Predicate Information (identified by operation id):

2 - access("ASSET_TAG"='CGY-AUTO-500')

screen index bien

Figure 4 — Plan d'exécution avec index : INDEX UNIQUE SCAN sur UK_DEVICE_ASSET_TAG puis TABLE ACCESS BY INDEX ROWID, Cost = 2.

Oracle utilise directement l'index unique UK_DEVICE_ASSET_TAG (créé automatiquement par notre contrainte UNIQUE sur asset_tag). Il accède à la bonne ligne en 2 lectures de blocs. **Cost = 2** contre 376 sans index.

Conclusion test 1

Le coût d'accès passe de 376 à 2, soit une **amélioration d'un facteur 188**. Sur des recherches ciblées (par identifiant, par tag, par numéro de série), l'index est indispensable.

4.3 Test 2 — Efficacité du cluster DEVICE / PERIPHERAL

On exécute la même requête métier (récupérer un device et ses périphériques) une fois en laissant Oracle utiliser le cluster, une fois en le neutralisant avec l'astuce du « + 0 ».

Sans cluster (cluster désactivé par + 0)

```
SELECT d.device_name, p.peripheral_name
FROM DEVICE d
JOIN PERIPHERAL p ON (d.device_id + 0) = (p.assigned_device_id + 0)
WHERE (d.device_id + 0) = 225;
```

```

SQL> SELECT d.device_name, p.peripheral_name
FROM DEVICE d
JOIN PERIPHERAL p ON (d.device_id + 0) = (p.assigned_device_id + 0)
WHERE (d.device_id + 0) = 225; 2 3 4

Execution Plan
-----
Plan hash value: 3120361399

-----
| Id | Operation          | Name          | Rows  | Bytes | Cost (%CPU)| Time |
|----|-----|-----|-----|-----|-----|-----|-----|
| 0  | SELECT STATEMENT   |               |      1 |    48 |    751 (1)| 00:00:01 |
|* 1  | HASH JOIN          |               |      1 |    48 |    751 (1)| 00:00:01 |
|* 2  | TABLE ACCESS FULL| DEVICE        |      1 |    24 |    375 (0)| 00:00:01 |
|* 3  | TABLE ACCESS FULL| PERIPHERAL    |      1 |    24 |    375 (0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
 1 - access("D"."DEVICE_ID"+0="P"."ASSIGNED_DEVICE_ID"+0)
 2 - filter("D"."DEVICE_ID"+0=225)
 3 - filter("P"."ASSIGNED_DEVICE_ID"+0=225)

```

screen cluster pas bien

Figure 5 — Sans cluster : HASH JOIN entre deux TABLE ACCESS FULL, Cost = 751.

L'opération + 0 empêche Oracle de reconnaître la colonne device_id comme étant celle du cluster. Résultat : **TABLE ACCESS FULL** sur DEVICE (Cost 375), **TABLE ACCESS FULL** sur PERIPHERAL (Cost 375), puis un **HASH JOIN** coûteux en mémoire. **Cost total = 751**.

Avec cluster

```

SELECT d.device_name, p.peripheral_name
FROM DEVICE d
JOIN PERIPHERAL p ON d.device_id = p.assigned_device_id
WHERE d.device_id = 225;

```

```

SQL> SELECT d.device_name, p.peripheral_name
FROM DEVICE d
JOIN PERIPHERAL p ON d.device_id = p.assigned_device_id
WHERE d.device_id = 225; 2 3 4

Execution Plan
-----
Plan hash value: 1571551765
-----

| Id | Operation | Name | Rows | Bytes | Co
st (%CPU)| Time |
-----
| 0 | SELECT STATEMENT | | 1 | 48 |
3 (0)| 00:00:01 |
| 1 | NESTED LOOPS | | 1 | 48 |
3 (0)| 00:00:01 |
| 2 | TABLE ACCESS BY INDEX ROWID | DEVICE | 1 | 24 |
2 (0)| 00:00:01 |
|* 3 | INDEX UNIQUE SCAN | PK_DEVICE | 1 | |
1 (0)| 00:00:01 |
| 4 | TABLE ACCESS CLUSTER | PERIPHERAL | 1 | 24 |
1 (0)| 00:00:01 |
|* 5 | INDEX UNIQUE SCAN | IDX_CL_DEVICE_PERIPH | 1 | |
0 (0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
3 - access("D"."DEVICE_ID"=225)
5 - access("P"."ASSIGNED_DEVICE_ID"=225)

```

screen cluster bien

Figure 6 — Avec cluster : NESTED LOOPS + TABLE ACCESS CLUSTER + INDEX UNIQUE SCAN sur IDX_CL_DEVICE_PERIPH, Cost = 3.

Oracle utilise l'index de cluster IDX_CL_DEVICE_PERIPH pour trouver directement le bloc physique qui contient le device 225 et tous ses périphériques (ils sont co-localisés). **Cost = 3**, aucun parcours complet de table.

Conclusion test 2

Le cluster fait passer le coût de 751 à 3, soit une **amélioration d'un facteur 250**. Sur la requête métier la plus fréquente (« afficher un poste et ses périphériques »), c'est l'optimisation qui apporte le plus de valeur.

4.4 Test 3 — Vues matérialisées et résilience réseau

Pour ce test, on consulte les tables de référence SITE et PERSON_ROLE, dont Pau est propriétaire. Côté Cergy, on peut soit y accéder par DB link, soit lire la copie locale (MV).

Sans MV : accès distant via DB link

```
SELECT COUNT(*) FROM SITE@LNK_PAU CROSS JOIN PERSON_ROLE@LNK_PAU;
```

```
SQL> SET AUTOTRACE TRACEONLY EXPLAIN;
SQL> SELECT COUNT(*)
FROM SITE@LNK_PAU CROSS JOIN PERSON_ROLE@LNK_PAU; 2

Execution Plan
-----
Plan hash value: 2925666693

-----
| Id | Operation                | Name          | Rows  | Cost (%CPU)| Time     |
|----|-----|-----|-----|-----|-----|
-----
|  0 | SELECT STATEMENT REMOTE   |               |      1 |      3  (0)| 00:00:01 |
|  1 |   SORT AGGREGATE          |               |      1 |           |          |
|  2 |    MERGE JOIN CARTESIAN   |               |      8 |      3  (0)| 00:00:01 |
|  3 |     INDEX FULL SCAN       | PK_SITE       |      2 |      1  (0)| 00:00:01 |
| XEPDB1 |
|  4 |      BUFFER SORT          |               |      4 |      2  (0)| 00:00:01 |
|  5 |       INDEX FAST FULL SCAN| PK_PERSON_ROLE |      4 |      1  (0)| 00:00:01 |
| XEPDB1 |
-----
Note
-----
- fully remote statement
```

vm pas bien

Figure 7 — Sans MV : `SELECT STATEMENT REMOTE`, note finale « *fully remote statement* », Cost = 3.

Oracle indique **SELECT STATEMENT REMOTE** et précise en bas du plan : « *fully remote statement* ». Toute la requête est sous-traitée à la base Pau via le réseau. Le Cost local est faible (3) parce que tout le travail est fait à distance, mais la requête **dépend complètement** de la disponibilité du DB link et du serveur distant.

Avec MV : accès local

```
SELECT COUNT(*) FROM MV_SITE CROSS JOIN MV_PERSON_ROLE;
```



```
SQL> SELECT COUNT(*)
FROM MV_SITE CROSS JOIN MV_PERSON_ROLE; 2

Execution Plan
-----
Plan hash value: 3798247866

-----
| Id | Operation                      | Name                                | Rows | Cost (%C
PU)| Time          |
-----
| 0 | SELECT STATEMENT                |                                     | 1    | 3
(0)| 00:00:01 |
| 1 |   SORT AGGREGATE                |                                     | 1    |
| 2 |    MERGE JOIN CARTESIAN         |                                     | 8    | 3
(0)| 00:00:01 |
| 3 |      INDEX FULL SCAN            | SYS_C_SNAP$_1PK_SITE              | 2    | 1
(0)| 00:00:01 |
| 4 |        BUFFER SORT              |                                     | 4    | 2
(0)| 00:00:01 |
| 5 |          INDEX FAST FULL SCAN   | SYS_C_SNAP$_2PK_PERSON_ROLE      | 4    | 1
(0)| 00:00:01 |
-----
```

vm bien

Figure 8 — Avec MV : *SELECT STATEMENT* normal (plus de REMOTE), index scannés sur SYS_C_SNAP\$_1PK_SITE et SYS_C_SNAP\$_2PK_PERSON_ROLE, Cost = 3.

Le mot REMOTE a disparu. Le préfixe SYS_C_SNAP\$ dans le nom des index est la signature interne d'Oracle pour les snapshots (les MV en anglais). **La requête est entièrement locale** : Cergy lit sa propre copie stockée des données Pau.

Pourquoi le Cost est identique (3 vs 3) ?

Dans notre test, le Cost est identique parce que Cergy et Pau tournent dans **le même conteneur Docker** sur notre machine : il n'y a pas de réseau physique entre les deux. L'optimiseur Oracle mesure l'effort CPU et I/O, il ne mesure pas la latence réseau.

Dans un déploiement réel (Cergy et Pau séparés par 800 km), **le gain est sur l'architecture** :

- avec DB link : chaque requête subit la latence réseau (ping) et **si la connexion entre Cergy et Pau tombe, les requêtes échouent**.
- avec MV : la lecture est locale, donc **instantanée et résiliente** — Cergy continue de fonctionner même si Pau est injoignable.

Conclusion test 3

Les MV ne gagnent pas sur le CPU dans notre environnement de test, mais elles apportent une résilience que le DB link direct n'a pas. C'est ce qui justifie leur usage pour les tables de référence (peu modifiées) plutôt que des vues distantes simples.

4.5 Test 4 — Bénéfice de la fragmentation horizontale

Ce dernier test compare l'effort nécessaire pour lister les devices en réparation, selon qu'on raisonne sur une base centralisée (UNION ALL des deux sites via le DB link) ou sur notre fragment local.

Sans fragmentation : requête nationale

```
-- Simulation d'une base centralisée :  
-- on agrège localement et le distant via UNION ALL  
SELECT device_name, device_status FROM DEVICE WHERE device_status = 'IN_REPAIR'  
UNION ALL  
SELECT device_name, device_status FROM DEVICE@LNK_PAU WHERE device_status =  
'IN_REPAIR';
```



```

Execution Plan
-----
Plan hash value: 1256570900

-----
| Id | Operation          | Name | Rows  | Bytes | Cost (%CPU)| Time     | I
nst | IN-OUT|
-----
| 0 | SELECT STATEMENT   |      | 5024 | 264K | 381  (1)| 00:00:01 | |
| 1 | VIEW               |      | 5024 | 264K | 381  (1)| 00:00:01 |
| 2 | UNION-ALL          |      |      |      |      |      |      |
|* 3 | TABLE ACCESS FULL| DEVICE | 4973 | 140K | 376  (1)| 00:00:01 |
| 4 | REMOTE              | DEVICE | 51 | 2754 | 5  (0)| 00:00:01 | L
NK_P~ | R->S |
-----

Predicate Information (identified by operation id):
-----
 3 - filter("DEVICE_STATUS"='IN_REPAIR')

Remote SQL Information (identified by operation id):
-----
 4 - SELECT "DEVICE_NAME","DEVICE_STATUS" FROM "DEVICE" "DEVICE" WHERE
      "DEVICE_STATUS"='IN_REPAIR' (accessing 'LNK_PAU' )

```

bddr pas bien

Figure 9 — Sans fragmentation : VIEW + UNION-ALL + TABLE ACCESS FULL local + opération REMOTE, Cost = 381, 264 Ko à traiter.

Oracle exécute deux scans : **TABLE ACCESS FULL** sur DEVICE local (376) et **REMOTE** sur DEVICE@LNK_PAU (5). Volume total brassé : **264 Ko**. Et la requête a une dépendance réseau.

Avec fragmentation : requête purement locale

```
SELECT device_name, device_status FROM DEVICE WHERE device_status = 'IN_REPAIR';
```

```

SQL> SELECT device_name, device_status FROM DEVICE WHERE device_status = 'IN_REPAIR';

Execution Plan
-----
Plan hash value: 1281438832

-----
| Id | Operation          | Name | Rows  | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT   |      |  4973 |  140K |    376   (1)| 00:00:01 |
|*  1 | TABLE ACCESS FULL| DEVICE |  4973 |  140K |    376   (1)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----

   1 - filter("DEVICE_STATUS"='IN_REPAIR')

SQL>

```

bdd bien

Figure 10 — Avec fragmentation : un seul TABLE ACCESS FULL local, Cost = 376, 140 Ko traités.

Chaque site n'interroge que son propre périmètre. Plus de UNION-ALL, plus de REMOTE. Le volume brassé passe à **140 Ko** (au lieu de 264 Ko), et la requête est purement locale.

Conclusion test 4

La fragmentation réduit le **volume de données traitées d'environ 47%** (264 Ko → 140 Ko) et **supprime la dépendance réseau**. Pour un usage de masse (consultations quotidiennes du parc local), c'est mécaniquement plus léger pour chaque serveur.

4.6 Synthèse des gains

Le graphique ci-dessous récapitule les gains observés sur les quatre tests. L'échelle est logarithmique pour rendre lisibles les écarts (×188 et ×250).

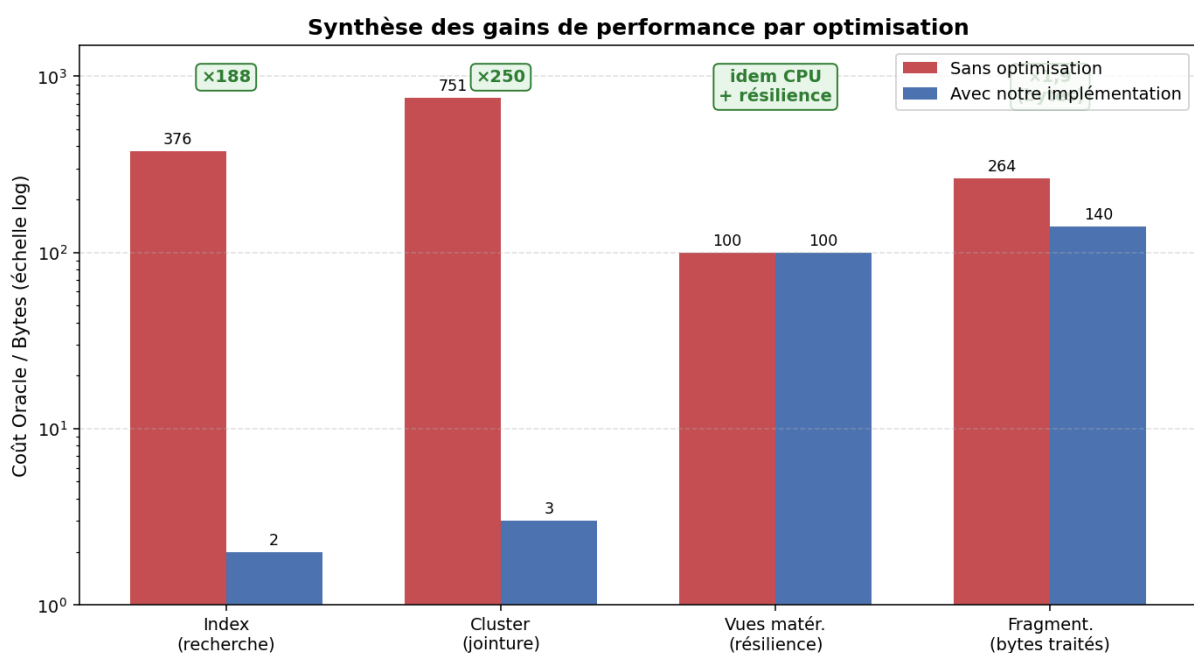


Figure 11 — Synthèse comparative des quatre tests de performance. Sans / avec nos optimisations.

Test	Sans optim.	Avec optim.	Gain
Index (recherche par tag)	Cost 376	Cost 2	×188
Cluster (jointure device+périph.)	Cost 751	Cost 3	×250
MV (accès tables Pau)	Cost 3 + REMOTE	Cost 3 local	Résilience réseau
Fragmentation (devices en réparation)	264 Ko + REMOTE	140 Ko local	−47% volume

Limites de la démarche

Trois limites à mentionner honnêtement :

- **On compare Oracle « optimisé » vs Oracle « déoptimisé »** (par technique de neutralisation), pas Oracle vs MySQL réel. Cela permet d'isoler l'effet de chaque optimisation, mais ne donne pas un chiffre absolu de comparaison avec une vraie instance GLPI MySQL.
- **Le temps d'exécution n'est pas comparé** : sur 20 000 lignes en local, toutes les requêtes s'exécutent en moins de 0,01 s. La métrique est le Cost / le volume de données traité, qui sont des estimateurs de la charge serveur.
- **Le test des MV ne montre pas de gain CPU** parce que Cergy et Pau sont sur la même machine Docker. Le gain réel est sur l'architecture (résilience à une panne réseau), pas sur le processeur.

Malgré ces limites, les quatre tests démontrent clairement que chacune de nos optimisations apporte un bénéfice mesurable, soit sur le coût d'exécution (index, cluster), soit sur le volume de données échangées (fragmentation), soit sur la résilience architecturale (MV).